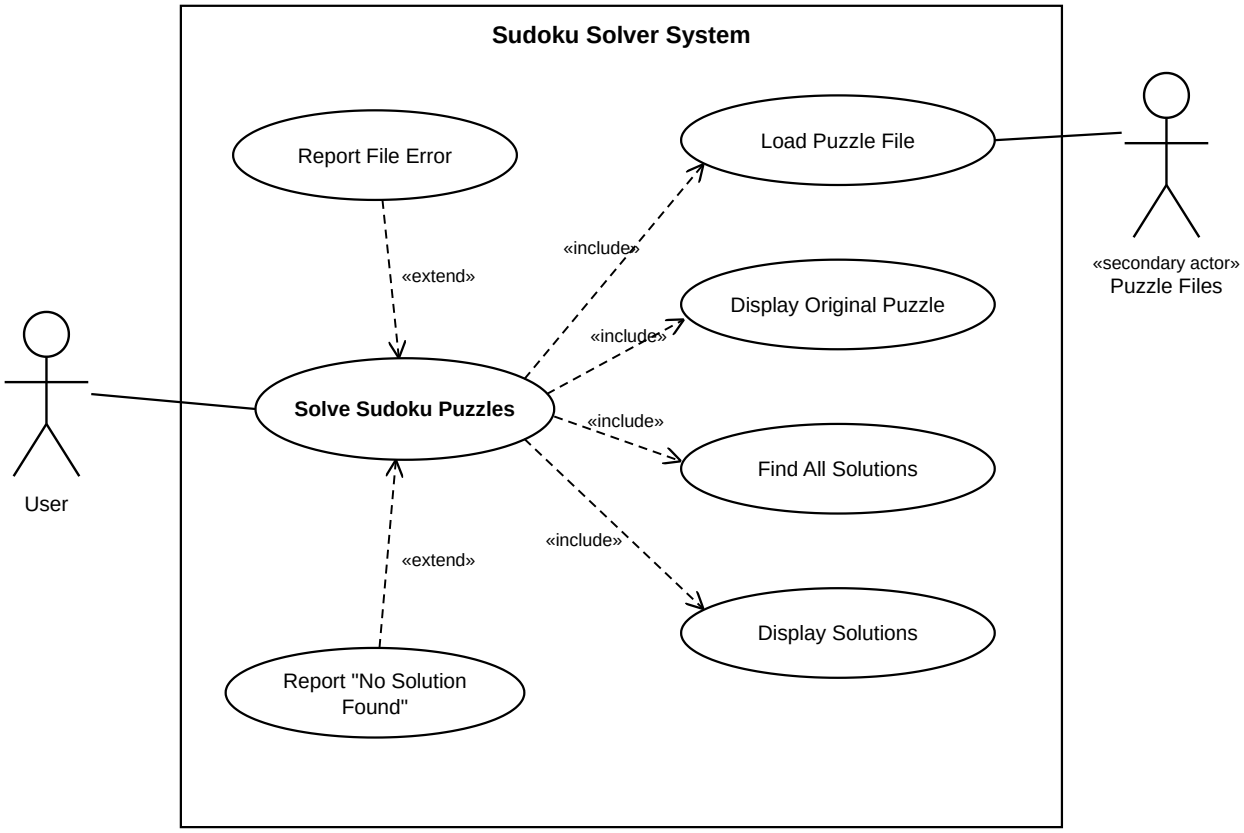

Sudoku Solver System
Report File Error
Load Puzzle File
«include»
«extend»
Display Original Puzzle
«include»
Solve Sudoku Puzzles
«include»
Find All Solutions
«include»
«extend»
Display Solutions
Report "No Solution Found"
User
«secondary actor»
Puzzle Files

1. Use Case

Solve Sudoku Puzzles. The main actor is the User who runs the program. The puzzle files (puzzle1.txt through puzzle5.txt) are a secondary actor because the system reads from them. This use case includes Load Puzzle File, Display Original Puzzle, Find All Solutions and Display Solutions. Report File Error and Report "No Solution Found" extend it.

2. Brief Description

The User runs the program and it goes through the five puzzle files one at a time. For each one it prints the file name and the puzzle, then uses recursion and backtracking to find every solution and prints them. If there is no solution or the file can't be read, it prints "No solution found".

3. Step-by-Step Flow

1. The User starts the program.
2. The system picks the next puzzle file.
3. The system prints the file name.
4. The system loads the puzzle from the file.
5. The system prints the original puzzle.
6. The system searches for every solution.
7. The system prints each solution and the total found.
8. Steps 2 through 7 repeat until all five files are done.

4. Detailed Description

a. Preconditions

The program is compiled on the cycle server and the five puzzle files are in the same folder.

b. Main Flow

The User runs the program. For each puzzle file the system makes a new solver object and prints the file name. It reads the 81 cells from the file, where a digit is a filled cell and an underscore is a blank, and checks that the file is valid. It prints the original puzzle. Then it picks a blank cell, finds which digits can go there without repeating in the row, column or block, and tries the first one. It calls itself again to fill in the next cell. When that call comes back, it removes the digit and tries the next one. When there are no blank cells left, it prints the solution and keeps going to find any others. When all choices have been tried, it prints the total number of solutions.

c. Alternate Flows

If a blank cell has no digits that fit, an earlier guess was wrong, so the call returns and the program backs up and tries something else. If no solutions are found, or the starting puzzle already repeats a digit, the program prints "No solution found". If the file can't be opened, has a

character other than 1-9 or an underscore, or doesn't have exactly 81 cells, the program prints an error and "No solution found", then moves on to the next file.

d. Postconditions

All five files have been printed with their file name, the original puzzle, and every solution or "No solution found". The puzzle files are not changed.